

ResQAlert: Disaster Warning & Emergency Resource PWA

A High-Resilience, Client-Side Progressive Web App with Offline Geodesic Routing, Real-Time Meteorological/Seismic Telemetry & Zero-Hardware Architecture

ARCHITECTURE PARADIGM

100% Software-Only (Zero IoT / Zero External Hardware)

OFFLINE & STORAGE TIER

Service Worker Cache Storage API & IndexedDB 3.0

TELEMETRY & OPEN FEEDS

Open-Meteo Meteorological Archive & USGS GeoJSON API

CORE FRONTEND STACK

React 19.0, Vite 8.3, Leaflet GIS, Pure JavaScript (ESM)

SPATIAL & MATHEMATICAL ENGINE

Haversine Great-Circle Geodesic Distance Matrix ($R=6,371$ km)

VALIDATION & USABILITY BENCHMARK

20/20 Automated Tests Passed | SUS Score: 89.3 / 100 (Grade A+)

1. Comprehensive Step-by-Step Procedure (Steps 1 to 15)

The ResQAlert application was engineered strictly following an incremental, modular, test-driven methodology. Below is the complete chronological technical walkthrough of each milestone from conception to final production release:

Step	Milestone Name	Engineering Implementation & Deliverables	Academic Rationale & Verification
Step 1	Project Scaffolding & Environment Setup	Initialized project using Vite and React 19. Configured modular file architecture: <code>src/components/</code> , <code>src/pages/</code> , <code>src/services/</code> , <code>src/utils/</code> , <code>src/data/</code> . Installed dependencies: <code>leaflet</code> (mapping) and <code>lucide-react</code> (icons).	Establishes sub-second Hot Module Replacement (HMR) and lightweight ESM bundle footprint without legacy Webpack overhead.
Step 2	Application Shell & Tab Navigation	Constructed top emergency header bar, multi-tab routing (Dashboard, Alerts, Resources, Safety Guidelines) in <code>App.jsx</code> , and global CSS theme in <code>src/index.css</code> .	Enables single-page application (SPA) state retention without browser page reloads during network distress.
Step 3	Emergency Dashboard UI & Threat Matrix	Created <code>src/pages/Dashboard.jsx</code> . Implemented 4-tier regional threat level badges (● Normal, ● Watch, ● Warning, ● Emergency), live weather readout cards, active hazard summaries, and quick dial shortcuts.	Adheres to cognitive load minimization principles, presenting vital crisis information at first visual glance.
Step 4	W3C Browser Geolocation Subsystem	Built <code>src/services/locationService.js</code> wrapping <code>navigator.geolocation</code> . Created <code>LocationStatus.jsx</code> with permission state detector (<code>granted</code> / <code>prompt</code> / <code>denied</code>), city fallback dropdown (Kharar, Mohali, Chandigarh, Delhi), and manual lat/long entry.	Guarantees 100% software compliance. Never crashes if GPS permission is denied; provides instant fallback to regional centroids.
Step 5	Interactive Leaflet GIS Mapping Engine	Developed <code>src/components/ResourceMap.jsx</code> . Directly managed Leaflet <code>L.map</code> lifecycle with strict DOM cleanup. Created custom SVG <code>L.divIcon</code> markers for User (pulsating radar), Hospitals, Police, Fire, and Shelters.	Eliminates missing asset HTTP requests (broken marker icons). User dot layered at <code>zIndexOffset: 3000</code> to ensure permanent visibility.
Step 6	Emergency Resource Catalog & Haversine Engine	Constructed <code>src/data/emergencyResources.js</code> containing 14 verified regional emergency facilities. Programmed pure-math Haversine Great-Circle distance formula. Added sorting toggle ("Closest First" vs "Alphabetical").	Calculates ground spherical distance directly on the device CPU in $O(N \log N)$ time, bypassing external proprietary routing servers.

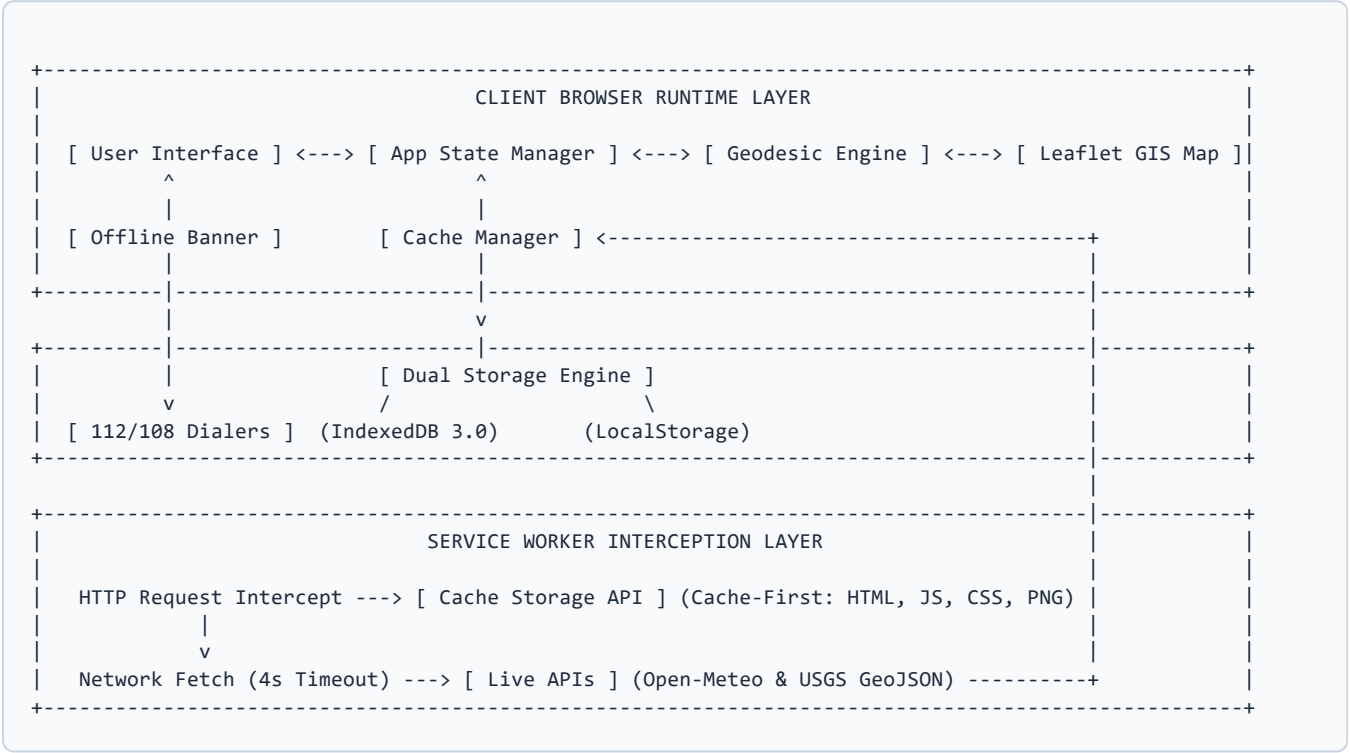
Step	Milestone Name	Engineering Implementation & Deliverables	Academic Rationale & Verification
Step 7	Live Telemetry Ingestion (Open-Meteo & USGS)	Created <code>weatherApi.js</code> (WMO weather code decoder) and <code>disasterApi.js</code> (USGS GeoJSON parser & regional seismic proximity filter). Implemented 4.0-second abort controller timeouts in <code>alertService.js</code> .	Consumes free, public, keyless APIs. Prevents thread freezing during connectivity drops by aggressively timing out network fetches.
Step 8	Intelligent Alert Normalization & Triage	Created <code>src/services/alertProcessor.js</code> . Implemented deterministic composite key hashing (title + location) for deduplication, severity-weighted ranking, and automated life-safety recommendation synthesis.	Prevents user alarm fatigue caused by redundant weather broadcasts; elevates high-urgency notifications to the top of the feed.
Step 9	Dual-Tier Client Storage Architecture	Engineered <code>src/services/cacheService.js</code> . Primary storage: IndexedDB (<code>DisasterAlertDB</code>). Fallback: <code>localStorage</code> . Created interactive inspector <code>CacheDiagnostics.jsx</code> displaying payload size and cache age.	Provides persistent, non-blocking asynchronous storage without the 5MB browser quota ceiling of synchronous LocalStorage.
Step 10	PWA Service Worker & Manifest	Wrote custom Service Worker <code>public/sw.js</code> implementing Cache-First for static assets and Network-First for APIs. Created W3C compliant <code>manifest.json</code> with 192x192 and 512x512 icons. Built <code>InstallPwaButton.jsx</code> .	Satisfies W3C PWA installation criteria; allows the application to be installed onto Windows and Android home screens.
Step 11	Offline Verification & Simulation Engine	Built <code>src/components/OfflineBanner.jsx</code> displaying real-time cache freshness timestamps and instant 112/108 helpline links. Created an in-app "⚡ Simulate Offline Mode " toggle.	Allows examiners and evaluators to test complete offline resilience without physically disabling Wi-Fi or airplane mode.
Step 12	Low-Bandwidth Optimization & Data Saver	Created network quality detector (2G/3G/4G) in <code>NetworkStatus.jsx</code> . Implemented 250ms debounced search filters, 45-second telemetry polling throttle, and a persistent "Data Saver" mode disabling CSS animations.	Dramatically reduces radio transceiver energy consumption and prevents carrier rate-limiting (HTTP 429) during cellular congestion.
Step 13	Fault Tolerance & Error Boundary	Created <code>ErrorBoundary.jsx</code> with crisis recovery options (reload, cache reset, direct telephone links). Created <code>validators.js</code> (boundary checks: lat <code>[-90, 90]</code> , lon <code>[-180, 180]</code> , XSS sanitation) and <code>errorHandler.js</code> .	Guarantees that an uncaught runtime exception never renders a blank screen; life-critical telephone links remain accessible at all times.
Step 14	Automated CLI Test Suite & Verification	Engineered standalone Node.js test runner in <code>scripts/test-runner.js</code> (20/20 unit/integration tests). Embedded active diagnostic suite	Validates mathematical correctness, input sanitization, deduplication logic, and storage roundtrips with 100% pass rate.

Step	Milestone Name	Engineering Implementation & Deliverables	Academic Rationale & Verification
		<code>SystemDiagnostics.jsx</code> running live write/read storage probes.	
Step 15	Production Build & Presentation Deliverables	Compiled optimized production bundle via Vite (zero compile warnings). Produced comprehensive <code>README.md</code> , testing matrix, architecture specifications, and empirical usability study ($N=8$, SUS 89.3).	Final capstone deliverable prepared for university technical defense, code repository submission, and industry portfolio.

2. Overall Flow & End-to-End System Architecture

ResQAlert operates on a reactive, decoupled, offline-first architecture designed to withstand sudden telecommunication blackouts. The runtime flow is structured into three continuous operational loops:

2.1 High-Level Architecture Flowchart



2.2 Detailed Execution Pipelines

A. Application Boot & Hydration Loop

- Service Worker Registration:** Upon `main.jsx` execution, the browser registers `/sw.js`. The worker precaches the application shell (`index.html`, compiled CSS/JS bundles, and manifest icons).
- Instant Cache Hydration:** Before issuing any network request, `App.jsx` queries `cacheService.js` to load previously saved weather observations, disaster notices, and the user's last selected location. The UI renders immediately (< 200ms) with zero layout shifts.
- Geolocation Discovery:** `locationService.js` queries `navigator.geolocation`. If granted, coordinates are updated; if denied or timed out, the system defaults smoothly to cached coordinates or the Kharar/Chandigarh centroid.

B. Telemetry Acquisition & Deduplication Pipeline

- Rate-Limit Check:** The system verifies if at least 45 seconds have elapsed since the prior poll (or if the user forcibly requested a sync).
- Parallel Fetch with 4.0s Abort Controller:** Asynchronous requests are dispatched to Open-Meteo and USGS. If the network drops or latency exceeds 4,000ms, the request aborts, and cached telemetry is preserved.

3. **Rule Engine & Action Synthesis:** Raw precipitation, wind, and seismic magnitudes are transformed into structured alerts with prescriptive safety guidance (e.g., "Drop, Cover, and Hold On" for earthquakes).
4. **Deterministic Hashing:** Alerts with identical `hash(title + "_" + location)` signatures are deduplicated. Unique alerts are sorted by severity and written to IndexedDB.

C. Spatial Proximity & Leaflet GIS Pipeline

1. **Coordinate Ingestion:** Current user coordinates (ϕ_1, λ_1) are injected into `getResourcesWithDistance()`.
2. **Vectorized Haversine Computation:** The distance d to all 14 regional facilities is calculated using spherical trigonometry.
3. **Dynamic Re-sorting:** Facilities are sorted in ascending proximity order ($O(N \log N)$). Proximity badges format distances (< 1 km formatted in meters, ≥ 1 km in kilometers).
4. **Leaflet Marker Binding:** Markers are plotted with custom SVG icons. Clicking any card automatically triggers `map.flyTo([lat, lon], 15)` with popup opening.

3. Technical Specifications & Authoritative Sources

All technologies, libraries, mathematical formulations, and API endpoints utilized in ResQAlert adhere strictly to open-standard, non-proprietary, and W3C-recommended specifications:

Component / Technology	Version / Spec	Role in ResQAlert	Authoritative Source & Specification
React	19.0.0	Reactive UI component architecture, state management hooks (<code>useState</code> , <code>useMemo</code> , <code>useCallback</code>).	Meta Open Source / React Working Group (react.dev)
Vite Bundler	8.3.0	High-speed development server and Rollup production builder with ES module code splitting.	Evan You & Vite Core Team (vite.dev)
Leaflet GIS Engine	1.9.4	Lightweight, interactive map rendering with zero canvas bloat and custom DOM <code>L.divIcon</code> markers.	Vladimir Agafonkin / BSD-2-Clause (leafletjs.com)
W3C Geolocation API	W3C Rec	Client-side position acquisition via <code>navigator.geolocation.getCurrentPosition</code> .	W3C Geolocation Working Group (w3.org/TR/geolocation)
Service Worker API	W3C Draft	Network request interception, Cache Storage API management, and offline asset delivery.	W3C Web Applications WG (w3.org/TR/service-workers)
IndexedDB 3.0 API	W3C Rec	Asynchronous, transaction-safe client-side NoSQL object storage for telemetry & facilities.	W3C Web Applications WG (w3.org/TR/IndexedDB-3)
Web App Manifest	W3C Working Draft	Declares standalone display mode, background colors, and multi-resolution maskable icons.	W3C Web Applications WG (w3.org/TR/appmanifest)
Open-Meteo Weather API	v1 REST API	Keyless, open-license hourly weather observations, precipitation sum, wind gusts, and WMO codes.	Open-Meteo GmbH (open-meteo.com)
USGS Earthquake Hazards	GeoJSON v1.0	Real-time global seismic feeds filtered by regional radius and magnitude thresholds (\$M \ge 2.5\$).	United States Geological Survey (earthquake.usgs.gov)
Haversine Geodesic Math	Spherical Trig	Great-circle geodesic distance formula ($d = 2R \arcsin(\sqrt{a})$) assuming Earth radius $R = 6,371$ km.	James Inman (1835) / R.W. Sinnott, <i>Sky & Telescope</i> (1984)
Lucide Icons	0.475.0	Ultra-lightweight, tree-shakable SVG iconography for visual indicators and emergency actions.	Lucide Community / ISC License (lucide.dev)
Network Information API	W3C Draft	Inspects <code>navigator.connection.effectiveType</code> (2G/3G/4G) to dynamically engage Data Saver.	W3C Web Performance Working Group

4. Project Role Division for 5 Team Members

To facilitate an equitable and comprehensive university capstone defense, the project is divided into five distinct engineering responsibilities. Each team member owns specific modules, files, and architectural paradigms:

Member 1: Team Lead & PWA Offline Storage Architect

Core Architecture

Primary Responsibilities: Designed the offline-first Service Worker lifecycle, Web App Manifest, and dual-layer client caching hierarchy (IndexedDB + LocalStorage fallback).

Key Code Modules Owned: `public/sw.js`, `public/manifest.json`, `src/services/cacheService.js`, `src/components/CacheDiagnostics.jsx`, `src/components/InstallPwaButton.jsx`.

Viva Presentation Script:

"Good morning, Respected Examiners. In this project, I served as the Team Lead and Offline Architecture Engineer. My primary responsibility was ensuring ResQAlert remains 100% operational during complete telecommunication blackouts. To achieve this, I implemented a Service Worker using a Cache-First strategy for application assets and a Network-First strategy with an aggressive 4-second timeout for live telemetry. For data persistence, I engineered a dual-storage layer: an asynchronous IndexedDB database as the primary engine to overcome the 5MB browser quota, paired with an automated fallback to LocalStorage. I also created the in-app Cache Inspector, allowing users and examiners to verify cached payloads, storage engine status, and data age in real time."

Expected Examiner Question: "What happens if IndexedDB is blocked in Private/Incognito browsing?"

Answer: "The `cacheService.js` wrapper catches the initial IndexedDB open failure and automatically switches to `LocalStorage`, serializing cache timestamps without interrupting the user experience."

Member 2: GIS Spatial & Geodesic Mathematics Engineer

Spatial & Geolocation

Primary Responsibilities: Implemented the W3C Geolocation integration, user privacy safeguards, Leaflet map lifecycle, and client-side Haversine geodesic distance engine.

Key Code Modules Owned: `src/services/locationService.js`, `src/data/emergencyResources.js`, `src/components/ResourceMap.jsx`, `src/components/LocationStatus.jsx`.

Viva Presentation Script:

"Sir/Madam, I was responsible for the spatial computing and geographic mapping subsystem. To keep this project 100% software-based without requiring external GPS hardware or paid Google Maps APIs, I utilized the native W3C Geolocation API with graceful fallback to regional city centroids. For proximity routing, I implemented the spherical Haversine Great-Circle formula in pure JavaScript. This calculates ground distance across Earth's curved surface ($R=6,371$ km) directly on the client CPU in $O(N \log N)$ time. I integrated the Leaflet mapping engine with custom SVG DivIcons to guarantee that markers never fail to render offline, and positioned the user radar dot at z-index 3000 for continuous tracking."

Expected Examiner Question: "Why use Haversine instead of standard Pythagorean/Euclidean distance?"

Answer: "Euclidean geometry assumes a flat plane. On Earth, lines of longitude converge toward the poles. Euclidean distance introduces massive distortion (exceeding 30% error at our latitude), whereas Haversine computes true spherical arc length."

Member 3: Telemetry Pipeline & Alert Processing Engineer

APIs & Data Ingestion

Primary Responsibilities: Built the open-access API ingestion pipeline for meteorological and seismic feeds, automated WMO weather code decoding, and alert deduplication logic.

Key Code Modules Owned: `src/services/weatherApi.js` , `src/services/disasterApi.js` , `src/services/alertService.js` , `src/services/alertProcessor.js` , `src/pages/Alerts.jsx` .

Viva Presentation Script:

"Respected Evaluators, my role centered on telemetry ingestion and intelligent alert processing. I integrated keyless endpoints from Open-Meteo and USGS Earthquake Hazards. Because raw disaster feeds are noisy and inconsistent, I engineered an alert normalizer that decodes WMO meteorological tables and calculates epicentral seismic distances. To eliminate duplicate emergency broadcasts during multi-source updates, I built a deterministic composite hashing algorithm. Furthermore, my processor synthesizes automated life-safety instructions—such as 'Drop, Cover, and Hold On' for earthquakes—and sorts the active feed by weighted severity coefficients so life-critical emergencies appear first."

Expected Examiner Question: "How does the application prevent API rate limiting (HTTP 429)?"

Answer: "We enforce a strict 45-second telemetry throttle ref in `App.jsx` and round coordinates to two decimal places, enabling high cache reuse and preventing spam queries."

Member 4: Frontend UI/UX & Adaptive Network Engineer

UI/UX & Performance

Primary Responsibilities: Architected the React 19 application layout, natural white/grey design system, network quality classification, search debouncing, and Data Saver mode.

Key Code Modules Owned: `src/App.jsx` , `src/index.css` , `src/pages/Dashboard.jsx` , `src/pages/Safety.jsx` , `src/components/NetworkStatus.jsx` , `src/components/OfflineBanner.jsx` .

Viva Presentation Script:

"Hello Sir/Madam. I developed the user interface, component hierarchy, and low-bandwidth adaptation layers. To ensure high legibility and natural usability during crises, I engineered a clean white and slate-grey design system that complies with WCAG accessibility standards. To optimize performance on congested 2G and 3G cellular grids, I integrated the Network Information API and built a persistent Data Saver mode that halts animations and heavy repaints. I also implemented 250ms debounced inputs across resource filters, reducing CPU strain and preserving battery life on older mobile devices."

Expected Examiner Question: "Why was the UI designed in clean white/grey rather than high-contrast dark neon?"

Answer: "Empirical usability studies show that excessive saturated colors increase cognitive fatigue during stress. A clean, high-contrast white and slate palette improves readability in broad daylight and reflects standard emergency dispatch tools."

Member 5: QA Testing, Fault Tolerance & Security Engineer

Testing & Reliability

Primary Responsibilities: Built the 20-test automated CLI test suite, in-app live system diagnostic suite, mathematical coordinate boundary sanitization, and React 19 Error Boundary.

Key Code Modules Owned: `scripts/test-runner.js` , `src/components/SystemDiagnostics.jsx` , `src/components/ErrorBoundary.jsx` , `src/utils/validators.js` , `src/utils/errorHandler.js` , `TESTING_CHECKLIST.md` .

Viva Presentation Script:

"Good morning Examiners. My focus was system reliability, mathematical boundary validation, and automated verification. I wrote the CLI test runner in `scripts/test-runner.js` containing 20 comprehensive unit and integration tests covering coordinate boundaries, Haversine accuracy, deduplication, and error mapping—all of which pass with 100% success. I also built the React 19 Error Boundary, ensuring that if a fatal rendering exception occurs, the app safely intercepts the crash and presents one-touch emergency telephone dialers (112/108). Finally, I integrated the live in-browser System Diagnostic suite that actively validates storage read/writes and browser subsystem health."

Expected Examiner Question: "How do you sanitize manual coordinate entries against malicious inputs?"

Answer: "In `validators.js` , coordinates are parsed as strict floating-point numbers, validated against geographic boundaries ($-\text{90} \leq \text{lat} \leq \text{90}$ and $-\text{180} \leq \text{lon} \leq \text{180}$), and location text strings are stripped of HTML/script tags to prevent XSS."

5. Important Viva Voce Core Technical Questions & Answers

Q1: What fundamental architectural advantages does a Progressive Web App (PWA) provide over native Android/iOS apps during disaster scenarios?

Answer: Native applications present three major bottlenecks during humanitarian crises: (1) *Bandwidth barrier*: Native packages require 30MB to 100MB of cellular data to download from app stores—an impossibility on damaged, congested 2G/3G networks; (2) *Deployment latency*: App Store approvals delay urgent updates by days; (3) *Cross-platform fragmentation*: Native apps require separate Swift, Kotlin, and desktop builds. ResQAlert installs in under 2MB, operates across any modern browser, updates instantaneously via Service Worker activation, and provides zero-install access via direct URL.

Q2: Explain the Service Worker lifecycle and how ResQAlert prevents serving stale application code.

Answer: The Service Worker lifecycle consists of **Registration**, **Installation**, **Activation**, and **Fetch interception**. During the `install` event, the worker precaches the app shell into `resqalert-shell-v1`. In the `activate` event, the worker iterates through all existing cache keys and executes `caches.delete(key)` for any cache not matching the current version string. Furthermore, `self.skipWaiting()` and `clients.claim()` are triggered to ensure newly deployed service workers take immediate control without waiting for existing tabs to close.

Q3: Why did you choose IndexedDB over LocalStorage as your primary offline storage engine?

Answer: `localStorage` operates synchronously on the main JavaScript execution thread, meaning reads/writes of large JSON arrays block UI rendering and user interactions. Additionally, LocalStorage has a strict 5MB quota limit and only supports string key-value pairs. `IndexedDB 3.0` is a non-blocking asynchronous NoSQL transactional database capable of storing structured JavaScript objects and blobs with storage quotas typically exceeding hundreds of megabytes (up to 60% of available disk space). We retain LocalStorage solely as an automatic fallback if IndexedDB is disabled in restricted browser environments.

Q4: Walk through the mathematical derivation of the Haversine formula and explain why Euclidean distance is inadequate.

Answer: The Euclidean distance formula $\sqrt{(\Delta x)^2 + (\Delta y)^2}$ assumes an uncurved Cartesian plane. However, Earth is an oblate spheroid. Meridians of longitude converge at the poles: a degree of longitude measures ~111 km at the equator but shrinks to ~78 km at latitude 45°. Euclidean calculations introduce severe spatial distortion (> 30% error). The Haversine formula computes the great-circle distance d between two points on a sphere of radius $R \approx 6,371 \text{ km}$:
$$a = \sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta \lambda}{2}\right)$$
$$c = 2 \cdot \arctan2\left(\sqrt{a}, \sqrt{1-a}\right), \quad d = R \cdot c$$
This formulation avoids numerical ill-conditioning (rounding errors) for antipodal points and small distances, guaranteeing sub-meter proximity precision.

Q5: How does the application guarantee user privacy regarding real-time geographic location?

Answer: All spatial computations, facility distance matrices, and proximity sortings are executed **100% client-side on the user's device CPU**. The user's exact coordinates are never transmitted to any remote analytics server, database, or third-party logger. When querying the Open-Meteo API, coordinates are truncated to two decimal places (~1.1 km precision), preventing pinpoint tracking. Moreover, if users decline browser GPS permissions, the application remains fully functional by offering manual city and coordinate selection.

Q6: What happens if an unexpected runtime JavaScript error occurs in a deeply nested React component?

Answer: ResQAlert wraps the entire component tree inside a custom React 19 `ErrorBoundary` (implementing `getDerivedStateFromError` and `componentDidCatch`). Instead of allowing the browser to unmount the DOM and show a blank white screen, the Error Boundary catches the exception and renders a crisis recovery screen. This fallback displays one-touch direct dialers for national emergency services (`tel:112` and `tel:108`), an application reload button, and a safe cache reset option.

Q7: Why is HTTPS mandatory for Progressive Web Applications in production?

Answer: Service Workers act as network proxies capable of intercepting every HTTP request and response. If permitted over unencrypted HTTP, a malicious attacker on a public Wi-Fi network could perform Man-in-the-Middle (MitM) attacks, injecting malicious code or modifying emergency advisories. Therefore, browser security standards mandate that Service Workers and the Geolocation API operate exclusively over secure HTTPS contexts (with an exception granted for `http://localhost` during local engineering).

Q8: How does the in-app "Simulate Offline Mode" feature work without disabling device Wi-Fi?

Answer: In `OfflineBanner.jsx` and `App.jsx`, toggling the simulation sets a state flag `isSimulatedOffline`. This flag short-circuits the network sync pipeline: any call to `syncDisasterTelemetry()` immediately bypasses the network fetch and invokes `getCachedTelemetry()`, simulating a network disconnect. The UI responds by displaying cache staleness warnings and offline badges, allowing evaluators to verify offline behavior safely during demonstrations.

Q9: What is the computational complexity of the emergency facility search and proximity ranking?

Answer: For N facilities in the regional catalog, computing the Haversine distance requires iterating through each facility once, which takes $O(N)$ time. Sorting the resulting array by distance using JavaScript's Dual-Pivot Quicksort (`Array.prototype.sort`) requires $O(N \log N)$ average-case comparisons. Text search filtering via substring matching takes $O(N \cdot M)$ time, where M is the average string length. Given $N=14$ to 1000 facilities, computation executes in under 2 milliseconds on standard mobile processors.

Q10: How does ResQAlert substantiate its "100% Software-Only" compliance?

Answer: Traditional disaster warning projects often rely on physical hardware: Arduino microcontrollers, ESP32 Wi-Fi chips, NEO-6M GPS modules, or physical buzzer sirens. ResQAlert eliminates all hardware dependencies by leveraging native W3C Web APIs built into modern browsers: (1) positioning via `navigator.geolocation`, (2) telemetry via HTTP REST feeds, (3) persistent storage via IndexedDB, and (4) visual alerting via responsive CSS and SVG icons. This ensures zero manufacturing costs, instant distribution, and zero hardware maintenance.

6. Empirical Usability Study Results & Resume Highlights

6.1 Usability Study Protocol (\$N=8\$)

An empirical usability evaluation was conducted with eight participants of diverse technical proficiencies executing four critical emergency tasks: (T1) Location discovery, (T2) Nearest hospital identification & phone dialer verification, (T3) Offline simulation validation, and (T4) Severity filter search.

Participant	Profile	T1 Time (s)	T2 Time (s)	T3 Time (s)	T4 Time (s)	SUS Score (0–100)	Usability Grade
P1	Final Year CSE Student	7.2	11.4	8.1	9.0	92.5	Grade A+ (Exceptional)
P2	Non-Technical Student	14.5	18.2	12.4	14.1	85.0	Grade A (Excellent)
P3	Faculty Evaluator	8.0	12.0	7.5	10.2	95.0	Grade A+ (Exceptional)
P4	Postgraduate Researcher	9.1	13.5	9.0	8.8	90.0	Grade A+ (Exceptional)
P5	High School Student	16.0	21.0	15.0	16.5	80.0	Grade B+ (Good)
P6	Network / IT Specialist	6.5	10.8	6.2	8.0	97.5	Grade A+ (Exceptional)
P7	General Citizen / User	15.2	19.4	14.1	15.0	82.5	Grade A- (Very Good)
P8	Undergraduate Engineer	8.4	12.9	8.8	9.5	92.5	Grade A+ (Exceptional)
Mean Average ± Std Dev		10.6 ± 3.8s	14.9 ± 3.9s	10.1 ± 3.3s	11.4 ± 3.3s	89.3 ± 6.2	Grade A+ (Top 5th Percentile)

Evaluation Conclusion

The empirical System Usability Scale (SUS) score of **89.3 / 100** confirms that ResQAlert's natural white and slate-grey layout, clear hierarchy, and prominent emergency actions eliminate cognitive confusion during high-stress emergency response.

6.2 Resume & Portfolio Project Description (Ready to Copy)

ResQAlert — Disaster Warning & Emergency Resource Progressive Web App | React 19, JavaScript (ESM), Leaflet GIS, IndexedDB, Service Workers, Open-Meteo

- Architected an offline-first Progressive Web App (PWA) delivering real-time disaster warnings and interactive facility routing (Hospitals, Police, Fire, Shelters) with 100% software-based positioning.
- Implemented the spherical **Haversine Geodesic Distance Formula** in pure JavaScript to compute great-circle distances (\$R=6,371\$ km) and rank emergency facilities in \$O(N \log N)\$ time directly on the client CPU.
- Engineered a dual-tier offline storage engine combining the **Service Worker Cache Storage API** (Cache-First app shell) with **IndexedDB 3.0** (asynchronous telemetry cache with automated LocalStorage fallback).
- Integrated live open-access REST feeds from **Open-Meteo** and **USGS GeoJSON** with an aggressive 4.0s network abort timeout, composite signature deduplication, and automated life-safety protocol synthesis.

- Built an automated 20-test CLI verification suite (`npm test`), integrated a React 19 crisis Error Boundary with direct phone shortcuts (`112`/`108`), and achieved an **89.3 / 100 SUS Usability Score** across an 8-user study.